

Program 1. Write a Python program to perform simple arithmetic operations
--

Aim:

To write a Python program that reads two numbers from the user and performs basic arithmetic operations such as addition, subtraction, multiplication, and division. Display the results using appropriate output formatting.

Program:

```
# Program: Simple Calculator for Two Numbers
#Read two numbers from the user
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

#Perform basic arithmetic operations and display the result
print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Quotient:", a / b)
```

Result:

Thus, the Python program to perform simple arithmetic operations was completed, and the output was verified successfully.

Program 2. Write a Python program to check whether a number is even or odd

Aim:

To write a Python program that takes an integer input from the user and checks whether the number is even or odd using conditional statements.

Program:

```
#Read one number from the user
num = int(input("Enter a number: "))

#Check if a number is even or odd
if num % 2 == 0:
    print(num, "is Even")
else:
    print(num, "is Odd")
```

Result:

Thus, the Python program to check whether a number is even or odd was completed, and the output was verified.

Program 3. Write a Python program to find the sum of the first N natural numbers using recursion.

Aim:

To write a Python program that defines a recursive function to calculate the sum of the first N natural numbers. Read the value of N from the user and display the final sum using the recursive function.

Program:

```
#Definition of Function
def recursive_sum(n):
    if n == 0:
        return 0
    else:
        return n + recursive_sum(n - 1)

#Get a number as a input from user
num = int(input("Enter a positive integer: "))

#Call the function and display the result
print("Sum of first", num, "natural numbers is:", recursive_sum(num))
```

Result:

Thus, the Python program to find the sum of the first N natural numbers using recursion was completed, and the output was verified successfully.

Program 4. Write a Python program to reverse a string
--

Aim:

To write a Python program that takes a string as input from the user and prints its reverse using slicing.

Program:

```
# Program to reverse a string using slicing
```

```
# Taking input from the user  
text = input("Enter a string: ")
```

```
# Reversing the string using slicing  
reversed_text = text[::-1]
```

```
# Displaying the result  
print("Reversed string:", reversed_text)
```

Result:

Thus, the Python program to reverse a string was completed and the output was verified successfully.

Program 5. Write a Python program to find the sum of the digits of a number
--

Aim:

Write a Python program that takes a positive integer as input and repeatedly sums its digits until the result is a single-digit number. Display the final result.

Program:

```
# Program to find the single-digit sum of a number

# Taking input from the user
num = int(input("Enter a positive integer: "))

# Loop to keep summing digits until a single-digit number is obtained
while num >= 10:
    temp = num
    digit_sum = 0
    while temp > 0:
        digit = temp % 10
        digit_sum += digit
        temp = temp // 10
    num = digit_sum

# Display the final result
print("Single-digit sum is:", num)
```

Result:

Thus, the Python program to find the sum of the digits of a number was completed and the output was verified successfully.

Program 6. Write a Python program to check whether a string is a palindrome or not

Aim:

To write a Python program that takes a string as input and checks whether it is a palindrome or not. The check should ignore case differences.

Program:

```
# Program to check whether a string is a palindrome

# Taking input from the user
text = input("Enter a string: ")

# Converting to lowercase for case-insensitive comparison
text = text.lower()

# Reversing the string
reversed_text = text[::-1]

# Checking for palindrome
if text == reversed_text:
    print("The string is a palindrome.")
else:
    print("The string is not a palindrome.")
```

Result:

Thus, the Python program to check whether a string is a palindrome or not was completed, and the output was verified successfully.

Program 7. Write a Python program to demonstrate the usage of bitwise operators

Aim:

To write a Python program that demonstrates the use of bitwise operators such as AND (&), OR (|), XOR (^), NOT (~), left shift (<<), and right shift (>>) on integer values. The program should take two integer inputs from the user and display the result of each operation.

Program:

```
# Program to demonstrate usage of bitwise operators in Python
```

```
# Taking two integer inputs from the user
```

```
a = int(input("Enter first integer (a): "))
```

```
b = int(input("Enter second integer (b): "))
```

```
# Bitwise AND
```

```
print("a & b =", a & b)
```

```
# Bitwise OR
```

```
print("a | b =", a | b)
```

```
# Bitwise XOR
```

```
print("a ^ b =", a ^ b)
```

```
# Bitwise NOT (for a)
```

```
print("~a =", ~a)
```

```
# Left Shift
```

```
print("a << 1 =", a << 1)
```

```
# Right Shift
```

```
print("a >> 1 =", a >> 1)
```

Result:

Thus, the Python program to demonstrate the usage of bitwise operators was completed and the output was verified successfully.

Program 8. Write a Python program to find the largest and smallest of three numbers
--

Aim:

To write a Python program that accepts three numbers as input from the user and determines the largest and the smallest among them using conditional statements. The program should display both the largest and smallest values.

Program:

```
# Program to find the largest and smallest among three numbers
```

```
# Taking input from the user
```

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
num3 = float(input("Enter third number: "))
```

```
# Finding the largest number
```

```
if (num1 >= num2) and (num1 >= num3):
```

```
    largest = num1
```

```
elif (num2 >= num1) and (num2 >= num3):
```

```
    largest = num2
```

```
else:
```

```
    largest = num3
```

```
# Finding the smallest number
```

```
if (num1 <= num2) and (num1 <= num3):
```

```
    smallest = num1
```

```
elif (num2 <= num1) and (num2 <= num3):
```

```
    smallest = num2
```

```
else:
```

```
    smallest = num3
```

```
# Displaying the results
```

```
print("Largest number is:", largest)
```

```
print("Smallest number is:", smallest)
```

Result:

Thus, the Python program to find the largest and smallest numbers among three was completed, and the output was verified successfully.

Program 9. Write a Python program to print 'n' terms of the Fibonacci series of numbers
--

Aim:

To write a Python program that accepts a number n from the user and prints the first n terms of the Fibonacci series. The Fibonacci series starts with 0 and 1, and each subsequent term is the sum of the previous two terms.

Program:

```
# Program to print 'n' terms of the Fibonacci series

# Taking input from the user
n = int(input("Enter the number of terms: "))

# Initializing the first two terms
a, b = 0, 1
count = 0

# Checking if the number of terms is valid
if n <= 0:
    print("Please enter a positive integer.")
elif n == 1:
    print("Fibonacci sequence up to", n, "term:")
    print(a)
else:
    print("Fibonacci sequence up to", n, "terms:")
    while count < n:
        print(a, end=' ')
        # Update values
        a, b = b, a + b
        count += 1
```

Result:

Thus, the Python program to print 'n' terms of the Fibonacci series of numbers was completed, and the output was verified successfully.

Program 10. Write a Python program to find the factorial of a number using Recursion

Aim:

To write a Python program that calculates the factorial of a non-negative integer using a recursive function. The program should prompt the user to enter a number, compute its factorial using recursion, and display the result.

Program:

```
# Recursive function to find the factorial of a number
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

# Taking input from the user
num = int(input("Enter a non-negative integer: "))

# Checking for valid input
if num < 0:
    print("Factorial is not defined for negative numbers.")
else:
    result = factorial(num)
    print("Factorial of", num, "is:", result)
```

Result:

Thus, the Python program to find the factorial of a number using recursion was completed, and the output was verified successfully.

Program 11. Write a Python program to find the number of occurrences of each vowel present in the given string

Aim:

To write a Python program that accepts a string from the user and counts the number of times each vowel (a, e, i, o, u) appears in the string. The program should display the individual count for each vowel.

Program:

```
# Program to count the number of occurrences of each vowel in a string

# Taking input from the user
text = input("Enter a string: ").lower()

# Initializing a dictionary to hold counts of vowels
vowel_count = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

# Looping through the string and updating the vowel counts
for char in text:
    if char in vowel_count:
        vowel_count[char] += 1

# Displaying the results
print("Vowel occurrences in the given string:")
for vowel, count in vowel_count.items():
    print(vowel, ":", count)
```

Result:

Thus, the Python program to find the number of occurrences of each vowel present in the given string was completed and the output was verified successfully.

Program 12. Write a program to create, concatenate, and print a string and access a substring from a given string

Aim:

To write a Python program to:

1. Accepts two strings from the user.
2. Concatenates and prints the result.
3. Prompts the user to enter start and end positions.
4. Displays the substring using slicing from the concatenated string.

Program:

```
# Program to create, concatenate, and print strings, and access a substring using user input
```

```
# Taking two strings from the user
```

```
str1 = input("Enter the first string: ")
```

```
str2 = input("Enter the second string: ")
```

```
# Concatenating the strings with a space
```

```
concatenated = str1 + " " + str2
```

```
print("Concatenated String:", concatenated)
```

```
# Taking start and end positions from the user for slicing
```

```
start = int(input("Enter the start index for substring: "))
```

```
end = int(input("Enter the end index for substring: "))
```

```
# Accessing and printing the substring
```

```
substring = concatenated[start:end]
```

```
print("Substring from index", start, "to", end, "is:", substring)
```

Result:

Thus, the Python program to create, concatenate, and print a string and access a substring from a given string was completed, and the output was verified successfully.

Program 13. Write a Python program to find whether the given pattern is available in the given string or not

Aim:

To write a Python program that takes a main string and a pattern as input from the user. The program should:

- Check whether the pattern is present in the main string.
- Count how many times the pattern appears.
- Display all starting index positions where the pattern is found.

Program:

```
# Program to search for a pattern, count its occurrences, and find positions
```

```
# Taking the main string input from the user
main_string = input("Enter the main string: ")
```

```
# Taking the pattern input
pattern = input("Enter the pattern to search: ")
```

```
# Checking if pattern exists
if pattern in main_string:
    print("Pattern found in the string.")
```

```
# Counting occurrences
count = main_string.count(pattern)
print("Number of occurrences:", count)
```

```
# Finding all positions
print("Pattern found at positions (starting index):", end=" ")
pos = -1
for i in range(count):
    pos = main_string.find(pattern, pos + 1)
    print(pos, end=" ")
```

```
else:
    print("Pattern not found in the string.")
```

Result:

Thus, the Python program to find whether the given pattern is available in the given string or not was completed, and the output was verified successfully.

Program 14. Write a Python program to check whether the given email ID is a valid Gmail ID or not

Aim:

To write a Python program that takes an email ID as input from the user and checks whether it is a valid **Gmail address**. A valid Gmail ID must:

- End with "@gmail.com"
- Contain only one "@" symbol
- Have a non-empty username before "@"

Program:

```
# Program to check whether an email ID is a valid Gmail ID

# Taking input from the user
email = input("Enter an email ID: ")

# Basic checks
if email.count("@") == 1 and email.endswith("@gmail.com"):
    username = email.split("@")[0]
    if username: # Check that there's something before '@'
        print("Valid Gmail ID.")
    else:
        print("Invalid Gmail ID: Username cannot be empty.")
else:
    print("Invalid Gmail ID.")
```

Result:

Thus, the Python program to check whether the given email ID is a valid Gmail ID or not was completed, and the output was verified successfully.

Program 15. Write a Python program to create a Lambda function to find the biggest of the given values

Aim:

To write a Python program that:

- Accepts three numbers from the user.
- Uses a **Lambda function** to find the **largest** (maximum) value among them.
- Displays the biggest value using the Lambda function result.

Program:

```
# Program to find the biggest of three numbers using a Lambda function

# Taking three numbers as input
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

# Lambda function to find maximum
max_value = (lambda x, y, z: x if (x >= y and x >= z) else (y if y >= z else z))(a, b, c)

# Displaying the result
print("The biggest value is:", max_value)
```

Result:

Thus, the Python program to create a Lambda function to find the biggest of given values was completed, and the output was verified successfully.

Program 16. Write a Python program to create, append, and remove elements from a list

Aim:

To write a Python program that:

1. Creates a list of elements using user input.
2. Appends a new element to the list.
3. Removes an element from the list.
4. Displays the list after each operation.

Program:

```
# Program to create, append, and remove elements from a list

# Step 1: Creating a list using user input
n = int(input("Enter the number of elements you want in the list: "))
my_list = []

for i in range(n):
    element = input(f"Enter element {i+1}: ")
    my_list.append(element)

print("Initial List:", my_list)

# Step 2: Appending a new element
new_element = input("Enter a new element to append: ")
my_list.append(new_element)
print("List after appending:", my_list)

# Step 3: Removing an element
remove_element = input("Enter an element to remove: ")
if remove_element in my_list:
    my_list.remove(remove_element)
    print("List after removing element:", my_list)
else:
    print(f"Element '{remove_element}' not found in the list.")
```

Result:

Thus, the Python program to create, append, and remove elements from a list was completed, and the output was verified successfully.

Program 17. Write a Python program to demonstrate working with tuples
--

Aim:

To write a Python program to:

1. Create a tuple with multiple data types.
2. Access elements using indexing and slicing.
3. Count the occurrences of a specific element.
4. Find the index of a given element in the tuple.

Program:

```
# Program to demonstrate working with tuples using user input
```

```
# Step 1: Taking input and creating a tuple
```

```
n = int(input("Enter the number of elements in the tuple: "))
temp_list = []
```

```
for i in range(n):
```

```
    element = input(f"Enter element {i+1}: ")
    temp_list.append(element)
```

```
# Convert list to tuple
```

```
my_tuple = tuple(temp_list)
print("Created Tuple:", my_tuple)
```

```
# Step 2: Accessing elements
```

```
print("Element at index 0:", my_tuple[0])
print("Element at last index:", my_tuple[-1])
```

```
# Step 3: Slicing the tuple
```

```
print("Tuple slice (from index 1 to 3):", my_tuple[1:4])
```

```
# Step 4: Count occurrences
```

```
item = input("Enter an element to count: ")
print(f"Number of occurrences of '{item}':", my_tuple.count(item))
```

```
# Step 5: Find index
```

```
if item in my_tuple:
    print(f"Index of first occurrence of '{item}':", my_tuple.index(item))
else:
    print(f"'{item}' not found in the tuple.")
```

Result:

Thus, the Python program to demonstrate working with tuples in Python was completed and the output was verified successfully.

Program 18. Write a Python program to demonstrate working with dictionaries

Aim:

To write a Python program that:

1. Creates a dictionary by accepting key-value pairs from the user.
2. Displays the entire dictionary.
3. Accesses and displays the value for a given key.
4. Updates a value for an existing key.
5. Deletes a key-value pair from the dictionary.

Program:

```
# Program to demonstrate working with dictionaries in Python
```

```
# Step 1: Creating a dictionary with user input
```

```
n = int(input("Enter the number of key-value pairs: "))
```

```
my_dict = {}
```

```
for i in range(n):
```

```
    key = input(f"Enter key {i+1}: ")
```

```
    value = input(f"Enter value for '{key}': ")
```

```
    my_dict[key] = value
```

```
# Step 2: Display the dictionary
```

```
print("\nCreated Dictionary:", my_dict)
```

```
# Step 3: Access value using a key
```

```
search_key = input("\nEnter a key to access its value: ")
```

```
if search_key in my_dict:
```

```
    print(f"Value for key '{search_key}':", my_dict[search_key])
```

```
else:
```

```
    print(f"Key '{search_key}' not found in the dictionary.")
```

```
# Step 4: Update a value
```

```
update_key = input("\nEnter a key to update its value: ")
```

```
if update_key in my_dict:
```

```
    new_value = input(f"Enter new value for '{update_key}': ")
```

```
    my_dict[update_key] = new_value
```

```
    print("Updated Dictionary:", my_dict)
```

```
else:
```

```
    print(f"Key '{update_key}' not found, cannot update.")
```

```
# Step 5: Delete a key-value pair
```

```
delete_key = input("\nEnter a key to delete from the dictionary: ")
```

```
if delete_key in my_dict:
```

```
    del my_dict[delete_key]
```

```
    print("Dictionary after deletion:", my_dict)
```

```
else:
```

```
    print(f"Key '{delete_key}' not found, cannot delete.")
```

Result:

Thus, the Python program to demonstrate working with dictionaries was completed and the output was verified successfully.

Program 19. Write a Python program to perform read and write operations on a file

Aim:

To Write a Python program to:

1. Create or open a text file.
2. Write some content to the file using user input.
3. Close the file after writing.
4. Reopen the same file and read its contents.
5. Display the contents to the user.

Program:

```
# Program to perform read and write operations on a file

# Step 1: Get filename from user
filename = input("Enter the filename to create/write (with .txt extension): ")

# Step 2: Writing to the file
with open(filename, 'w') as file:
    print("Enter text to write to the file (type 'STOP' to finish):")
    while True:
        line = input()
        if line.upper() == 'STOP':
            break
        file.write(line + '\n')

print(f"\nData written successfully to '{filename}'.")

# Step 3: Reading from the file
print(f"\nReading contents of '{filename}':")
with open(filename, 'r') as file:
    content = file.read()
    print(content)
```

Result:

Thus, the Python program to perform read and write operations on a file was completed, and the output was verified successfully.

Program 20. Write a Python program to copy the contents of one file to another file

Aim:

To write a Python program that:

- Reads content from an existing source text file.
- Creates or overwrites a destination file.
- Copies the content from the source file to the destination file.
- Displays the contents of both files before and after copying.

Program:

```
# Program to copy contents from one file to another and display contents
```

```
# Step 1: Input source and destination file names
```

```
source_file = input("Enter the name of the source file (with .txt extension): ")
```

```
destination_file = input("Enter the name of the destination file (with .txt extension): ")
```

```
try:
```

```
    # Step 2: Open the source file in read mode and read content
```

```
    with open(source_file, 'r') as src:
```

```
        content = src.read()
```

```
        print(f"\n--- Contents of '{source_file}' ---")
```

```
        print(content)
```

```
    # Step 3: Write content to destination file
```

```
    with open(destination_file, 'w') as dest:
```

```
        dest.write(content)
```

```
    # Step 4: Read and display contents of the destination file
```

```
    with open(destination_file, 'r') as dest:
```

```
        copied_content = dest.read()
```

```
        print(f"\n--- Contents of '{destination_file}' after copying ---")
```

```
        print(copied_content)
```

```
    print(f"\n Contents successfully copied from '{source_file}' to '{destination_file}'.")
```

```
except FileNotFoundError:
```

```
    print(f"\n Error: The file '{source_file}' does not exist.")
```

```
except Exception as e:
```

```
    print(f"\n An unexpected error occurred: {e}")
```

Result:

Thus, the Python program to copy the contents of one file to another file was completed successfully, and the output was verified successfully.